commons-cli-scope.txt
TXT 149.65KB

You are a code reviewer. The attached file "commons-cli-scope.txt" contains the Java source code of the Apache Commons CLI library (34 files, comments removed). Each file starts with a line "// ===== FILE: <path> =====".

Analyze the code for exactly these three code smells and ignore all others:

Long Method

- Smell: Long Method
  - Code unit: `DefaultParser.parse(Options, Properties, NonOptionAction, String...)`
  - File: org/apache/commons/cli/DefaultParser.java
  - Evidence: Allocates `expectedOpts`, resets every `OptionGroup.setSelected(null)`, builds `cmd = CommandLine.builder()...get()`, iterates `for (final String argument : arguments) { handleToken(argument); }`, then `checkRequiredArgs()`, `handleProperties(properties)`, and `checkRequiredOptions()`.
  - Explanation: The method bundles parsing setup, per-token dispatch, group reset, properties handling, and validation into a single body, so it does several distinct things at once.
- Smell: Long Method
  - Code unit: `DefaultParser.handleShortAndLongOption(String)`
  - File: org/apache/commons/cli/DefaultParser.java
  - Evidence: Multi-level `if/else` over `token.length() == 1`, `pos == -1`, `options.hasShortOption`, `!getMatchingLongOptions(token).isEmpty()`, `getLongPrefix`, `isJavaProperty`, `handleConcatenatedOptions`, then the `else` branch with `opt.length() == 1`, `isJavaProperty(opt)`, and `handleLongOptionWithEqual`.
  - Explanation: A single method encodes many alternative token shapes (single char, short, long, `=`, Java property, concatenated), making the control flow long and hard to follow.
- Smell: Long Method

- Code unit:

  `HelpFormatter.printHelp(PrintWriter, int, String, String, Options, int, int, String, boolean)`

- File: org/apache/commons/cli/HelpFormatter.java
- Evidence: Validates `cmdLineSyntax`, branches on `autoUsage` to call `printUsage(...)` two ways, then conditionally `printWrapped(header)`, `printOptions(...)`, and conditionally `printWrapped(footer)`.
- Explanation: The method performs validation, usage rendering, header rendering, options rendering, and footer rendering in one place.

- Smell: Long Method
  - Code unit: `HelpFormatter.printUsage(PrintWriter, int, String, Options)`
  - File: org/apache/commons/cli/HelpFormatter.java
  - Evidence: Builds `StringBuilder buff`, tracks `processedGroups`, sorts `optList`, loops `for (final Iterator<Option> it = optList.iterator(); it.hasNext();)`, looks up `options.getOptionGroup(option)`, appends `appendOptionGroup(buff, group)` or `appendOption(buff, option, option.isRequired())`, adds spaces, then calls `printWrapped`.
  - Explanation: Usage-string construction, group de-duplication, sorting, and output are all combined into one method.
- Smell: Long Method
  - Code unit: `HelpFormatter.appendOptions(A, int, Options, int, int)`
  - File: org/apache/commons/cli/HelpFormatter.java
  - Evidence: Computes `maxSince`, builds `prefixList`, loops over `options.helpOptions()` composing `optBuf` with opt/long-opt/arg-name, computes `nextLineTabStop`, builds the `optHeader` when `showSince`, then a second loop appends since/padding/ `deprecatedFormatFunction` /description and calls `appendWrappedText`.
  - Explanation: The method does layout measurement, prefix construction, header construction, and body rendering, all in one large body.
- Smell: Long Method
  - Code unit: `HelpFormatter.findWrapPos(String, int, int)`
  - File: org/apache/commons/cli/HelpFormatter.java

- Evidence: Checks `indexOf(Char.LF, startPos)` and `indexOf(Char.TAB, startPos)`, returns `-1` on `startPos + width >= text.length()`, loops `for (pos = startPos + width; pos >= startPos; --pos)` scanning for `Char.SP`, `Char.LF`, `Char.CR`, then falls back to `pos = startPos + width`.
- Explanation: Multiple search strategies and fallback branches are chained into a single method.

- Smell: Long Method
  - Code unit: `GnuParser.flatten(Options, String[], boolean)`
  - File: org/apache/commons/cli/GnuParser.java
  - Evidence: Nested `if/else` chain for `--`, `-`, `arg.startsWith("-")`, `options.hasOption(opt)`, `equalPos != -1`, `options.hasOption(arg.substring(0, 2))`, `eatTheRest = stopAtNonOption`, plus inner `for (i++; i < arguments.length; i++)` loop.
  - Explanation: The method mixes token classification, option lookup, `=` splitting, two-character prefix splitting, and remaining-token consumption in one body.
- Smell: Long Method
  - Code unit: `PosixParser.flatten(Options, String[], boolean)`
  - File: org/apache/commons/cli/PosixParser.java
  - Evidence: Branches on `--` / `-`, `token.startsWith("--")` with `getMatchingOptions` / `AmbiguousOptionException`, `token.startsWith("-")` with `token.length() == 2`, `options.hasOption`, `getMatchingOptions`, `burstToken`, and finally `processNonOptionToken`, all inside a `while (iter.hasNext())`.
  - Explanation: Token splitting, long-option matching, ambiguity detection, and bursting are all performed in one method.
- Smell: Long Method
  - Code unit: `Parser.parse(Options, String[], Properties, boolean)`
  - File: org/apache/commons/cli/Parser.java
  - Evidence: Clears values, resets groups, calls `flatten(...)`, iterates `ListIterator` with branches for `"--"`, `"-"`, `token.startsWith("-")`, `stopAtNonOption`, calls `processOption(token, iterator)`, `cmd.addArg`, and `processProperties(properties)`, then `checkRequiredOptions()`.
  - Explanation: Setup, flattening, token loop with multiple modes, properties processing, and validation are all combined in one method.

- Smell: Long Method
  - Code unit: `PatternOptionBuilder.parsePattern(String)`
  - File: org/apache/commons/cli/PatternOptionBuilder.java
  - Evidence: Loop `for (int i = 0; i < pattern.length(); i++)`, branch on `isValueCode(ch)`, `ch == '!'`, otherwise `getValueType(ch)`, builds `TypeHandler` map, mutates `opt/required/type/converter`, flushes pending option, and builds a final trailing option.
  - Explanation: Pattern scanning, state mutation, converter-map construction, and option building are done together.
- Smell: Long Method
  - Code unit: `TextHelpAppendable.adjustTableFormat(TableDefinition)`
  - File: org/apache/commons/cli/help/TextHelpAppendable.java
  - Evidence: First loop over `table.columnTextStyles()` with header-width and per-row cell-width updates, then `calcWidth` / `adjustedMaxWidth` computation, a second loop over `styleBuilders` with `resize(...)`, and final `TableDefinition.from(...)`.
  - Explanation: The method adjusts min/max widths, computes available width, scales columns, and rebuilds the table in one body.

God Class

- Smell: God Class
  - Code unit: `CommandLine`
  - File: org/apache/commons/cli/CommandLine.java
  - Evidence: Large getter surface (`getArgList`, `getArgs`, `getOptionCount`, `getOptionObject`, `getOptionProperties`, `getOptions`, many `getOptionValue` / `getOptionValues` overloads), many `getParsedOptionValue(s)` overloads for `char` / `Option` / `OptionGroup` / `String`, plus `hasOption` overloads, `iterator`, `resolveOption`, `processPropertiesFromValues`, and `handleDeprecated`.
  - Explanation: One class handles arg storage, option lookup, typed parsing, properties extraction, deprecation callbacks, and iteration, which is a very broad set of responsibilities.
- Smell: God Class
  - Code unit: `DefaultParser`

- File: org/apache/commons/cli/DefaultParser.java
- Evidence: Fields `cmd`, `options`, `stopAtNonOption`, `nonOptionAction`, `currentToken`, `currentOption`, `skipParsing`, `expectedOpts`; helpers `checkRequiredArgs`, `checkRequiredOptions`, `getLongPrefix`, `getMatchingLongOptions`, `handleConcatenatedOptions`, `handleLongOption`, `handleLongOptionWithEqual`, `handleLongOptionWithoutEqual`, `handleOption`, `handleProperties`, `handleShortAndLongOption`, `handleToken`, `handleUnknownToken`, `isArgument`, `isJavaProperty`, `isLongOption`, `isNegativeNumber`, `isOption`, `isShortOption`, `updateRequiredOptions`, and multiple `parse` overloads.
- Explanation: The parser owns token classification, option matching, group/required-option bookkeeping, properties handling, and `CommandLine` construction all at once.

- Smell: God Class
  - Code unit: `HelpFormatter`
  - File: org/apache/commons/cli/HelpFormatter.java
  - Evidence: Public fields `defaultWidth`, `defaultLeftPad`, `defaultDescPad`, `defaultSyntaxPrefix`, `defaultNewLine`, `defaultOptPrefix`, `defaultLongOptPrefix`, `defaultArgName`; methods `appendOption`, `appendOptionGroup`, `appendOptions`, `appendWrappedText`, `createPadding`, `determineMaxSinceLength`, `findWrapPos`, many `printHelp` / `printOptions` / `printUsage` / `printWrapped` overloads, `renderOptions`, `renderWrappedText`, `renderWrappedTextBlock`, `rtrim`, and many setters.
  - Explanation: It handles option-prefix formatting, usage generation, wrapping, padding, sorting, deprecation formatting, and `PrintWriter` output, which is a wide set of responsibilities.
- Smell: God Class
  - Code unit: `Option`
  - File: org/apache/commons/cli/Option.java
  - Evidence: Fields for identity (`option`, `longOption`), typing (`type`, `converter`), metadata (`description`, `deprecated`, `since`), arity (`argCount`, `optionalArg`, `valueSeparator`), and values (`values`), plus methods `acceptsArg`, `add`, `clearValues`, `clone`, `getConverter`, `getKey`, `getValue(s)`, `hasArg`, `hasArgs`, `hasOptionalArg`, `hasValueSeparator`, `processValue`, `requiresArg`, many setters, `toDeprecatedString`, and `toString`.
  - Explanation: The same class holds configuration, value storage, conversion, deprecation, and comparison/printing logic, giving it many responsibilities and

heavy data access.

- Smell: God Class
  - Code unit: `TextHelpAppendable`
  - File: org/apache/commons/cli/help/TextHelpAppendable.java
  - Evidence: Methods `indexOfWrap`, `adjustTableFormat`, `appendHeader`, `appendList`, `appendParagraph`, `appendTable`, `appendTitle`, `makeColumnQueue`, `makeColumnQueues`, `printQueue`, `printWrapped`, `resize`, setters `setIndent`, `setLeftPad`, `setMaxWidth`, and `writeColumnQueues`.
  - Explanation: It implements heading/list/paragraph/table rendering, wrapping, column layout, indentation state, and scaling, which are several distinct responsibilities in one class.

Long Parameter List

- Smell: Long Parameter List
  - Code unit:

    ```
    HelpFormatter.printHelp(PrintWriter, int, String, String, Options, int, int, String, boolean)
    ```

  - File: org/apache/commons/cli/HelpFormatter.java
  - Evidence: Nine parameters: `pw`, `width`, `cmdLineSyntax`, `header`, `options`, `leftPad`, `descPad`, `footer`, `autoUsage`.
  - Explanation: Nine parameters exceed a comfortable threshold and force callers to track positional layout and output arguments together.
- Smell: Long Parameter List
  - Code unit:

    ```
    HelpFormatter.printHelp(PrintWriter, int, String, String, Options, int, int, String)
    ```

  - File: org/apache/commons/cli/HelpFormatter.java
  - Evidence: Eight parameters: `pw`, `width`, `cmdLineSyntax`, `header`, `options`, `leftPad`, `descPad`, `footer`.
  - Explanation: Eight positional parameters make calls error-prone and hard to read.
- Smell: Long Parameter List
  - Code unit: `HelpFormatter.printHelp(int, String, String, Options, String, boolean)`
  - File: org/apache/commons/cli/HelpFormatter.java

- Evidence: Six parameters: `width`, `cmdLineSyntax`, `header`, `options`, `footer`, `autoUsage`.
- Explanation: Six parameters combine layout, syntax, header/footer, options, and mode selection in one signature.

- Smell: Long Parameter List
  - Code unit: `HelpFormatter.printHelp(int, String, String, Options, String)`
  - File: org/apache/commons/cli/HelpFormatter.java
  - Evidence: Five parameters: `width`, `cmdLineSyntax`, `header`, `options`, `footer`.
  - Explanation: Five positional parameters still make the call site opaque about which string is header versus footer.
- Smell: Long Parameter List
  - Code unit: `HelpFormatter.appendOptions(A, int, Options, int, int)`
  - File: org/apache/commons/cli/HelpFormatter.java
  - Evidence: Five parameters: `sb`, `width`, `options`, `leftPad`, `descPad`.
  - Explanation: The append target, width, option set, and two padding values are passed positionally, making the call hard to interpret.
- Smell: Long Parameter List
  - Code unit: `DefaultParser.parse(Options, Properties, NonOptionAction, String...)`
  - File: org/apache/commons/cli/DefaultParser.java
  - Evidence: Parameters `options`, `properties`, `nonOptionAction`, and varargs `arguments`.
  - Explanation: Four parameters combine configuration, properties source, behavior mode, and arguments, so the signature carries several unrelated concerns.
- Smell: Long Parameter List
  - Code unit: `Option.Builder` constructor is not long, but `Option(String, String, boolean, String)` is the constructor with the longest list
  - Code unit: `Option(String, String, boolean, String)`
  - File: org/apache/commons/cli/Option.java
  - Evidence: Constructor parameters `option`, `longOption`, `hasArg`, `description`.
  - Explanation: Four positional constructor parameters with two adjacent strings make accidental swaps (opt vs long opt) easy.
- Smell: Long Parameter List

- Code unit: `Options.addRequiredOption(String, String, boolean, String)`
- File: org/apache/commons/cli/Options.java
- Evidence: Parameters `opt`, `longOpt`, `hasArg`, `description`.
- Explanation: Four positional parameters, including two strings and a flag, make the call site hard to read.

- Smell: Long Parameter List
  - Code unit: `Options.addOption(String, String, boolean, String)`
  - File: org/apache/commons/cli/Options.java
  - Evidence: Parameters `opt`, `longOpt`, `hasArg`, `description`.
  - Explanation: The same four-positional-parameter shape is repeated, increasing the chance of argument-order mistakes.
- Smell: Long Parameter List
  - Code unit: `Options.addOption(String, boolean, String)`
  - File: org/apache/commons/cli/Options.java
  - Evidence: Parameters `opt`, `hasArg`, `description`.
  - Explanation: Three parameters with a boolean in the middle force callers to remember positional meaning.